

Phân Tích Cú Pháp (Dependency Parsing)

Xử Lý Ngôn Ngữ Tự Nhiên
(Natural Language Processing - NLP)
Vũ Đình Khôi

Nội dung

- Khái niệm
- Các quan hệ phụ thuộc (Dependency Relations)
- Các hình thức phụ thuộc (Dependency Formalisms)
- Ngân hàng câu được phân giải cú pháp (Dependency Treebanks)
- Phân tích cú pháp dựa trên chuyển trạng thái (Transition-Based Dependency Parsing)
- Phân tích cú pháp dựa trên đồ thị (Graph-Based Dependency Parsing)

- [1] Chapter 19. Dan Jurafsky and James H. Martin, *Speech and Language Processing*, 3rd ed. Pearson, 2026.
 - <https://web.stanford.edu/~jurafsky/slp3/>
 - [2] Sowmya Vajjala, Bodhisattwa Majumder, Anuj Gupta, and Harshit Surana, *Practical Natural Language Processing*. O'Reilly, 2020.
 - [3] Jay Alammam and Maarten Grootendorst, *Hands-On Large Language Models*. O'Reilly, 2024.
 - [4] Suhas Pai, *Designing Large Language Model Applications*. O'Reilly, 2025.
 - Sinh viên đọc thêm:
 - Ngữ pháp phi-ngữ cảnh và Phân tích cú pháp thành phần (Context-Free Grammars and Constituency Parsing)
 - [1] Chapter 18. Dan Jurafsky and James H. Martin, *Speech and Language Processing*, 3rd ed. Pearson, 2026.
- Chỉ cần đọc các Chapter được liệt kê*

I find that I don't understand things unless I try to program them.



Donald E. Knuth

Professor Emeritus at Stanford University

If you want to **master something**, teach it



-- Richard Feynman

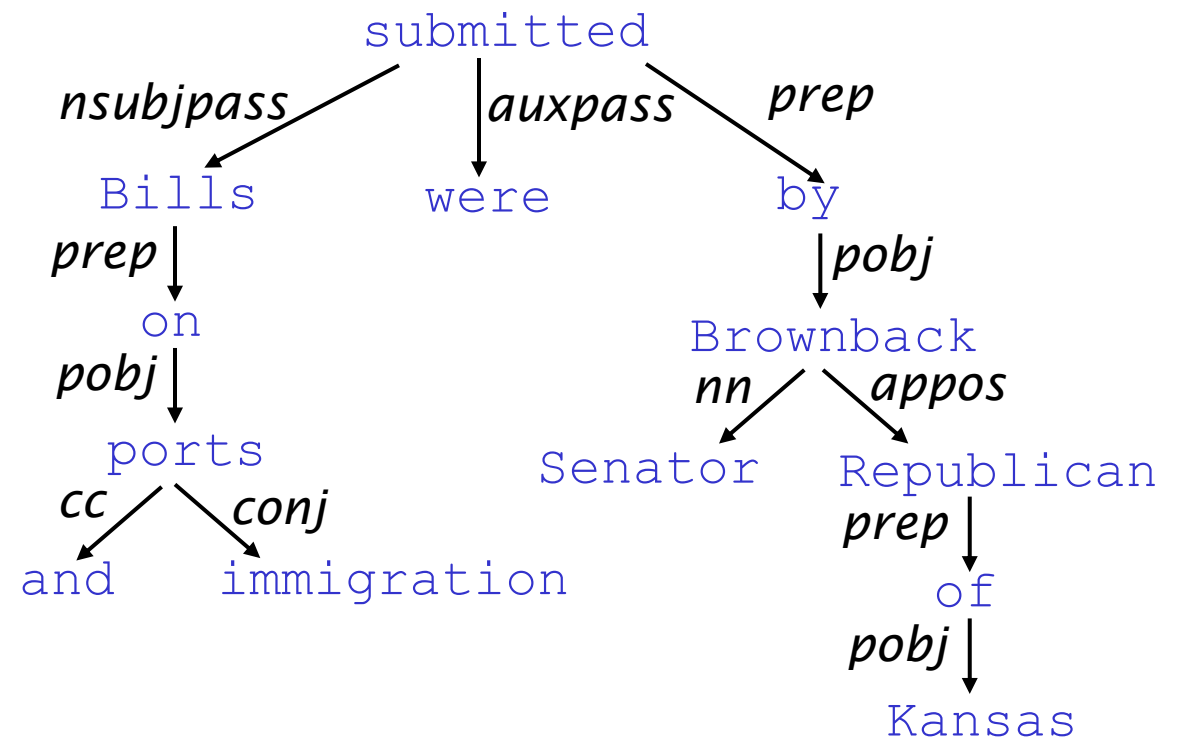
Ngữ pháp phụ thuộc (Dependency Grammar) và Cấu trúc phụ thuộc (Dependency Structure)

- Lý thuyết cú pháp phụ thuộc (**Dependency syntax**) cho rằng cấu trúc cú pháp (**syntactic structure**) bao gồm các thành phần từ vựng (**lexical items**) được liên kết bởi các mối quan hệ bất đối xứng nhị phân ("**mũi tên**") được gọi là các phụ thuộc (**dependencies**).

Các **mũi tên** thường được đặt tên theo các mối quan hệ ngữ pháp như chủ ngữ (**subject**), tân ngữ giới từ (**prepositional object**), bổ ngữ (**apposition**)...

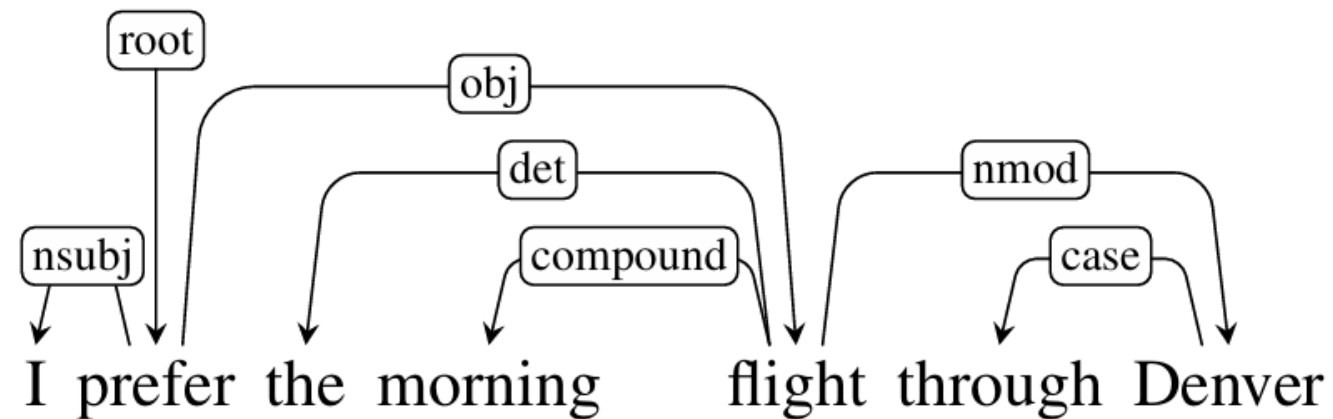
Mũi tên nối một từ chính **head** (governor, superior, regent) với một từ bổ trợ cho nó/từ phụ thuộc **dependent** (modifier, inferior, subordinate)

Thường các quan hệ phụ thuộc (**dependencies**) tạo thành một cấu trúc cây **tree** (liên thông /connected, không có chu trình/acyclic, chỉ có một head/single-head)



Dependency Grammar and Dependency Structure

▪ Phân tích sự phụ thuộc (dependency parse)



- Từ “flight” phụ thuộc vào/bổ trợ cho động từ “prefer”
- Từ “morning” bổ trợ cho danh từ “flight”

Dependency Grammar and Constituency Grammar

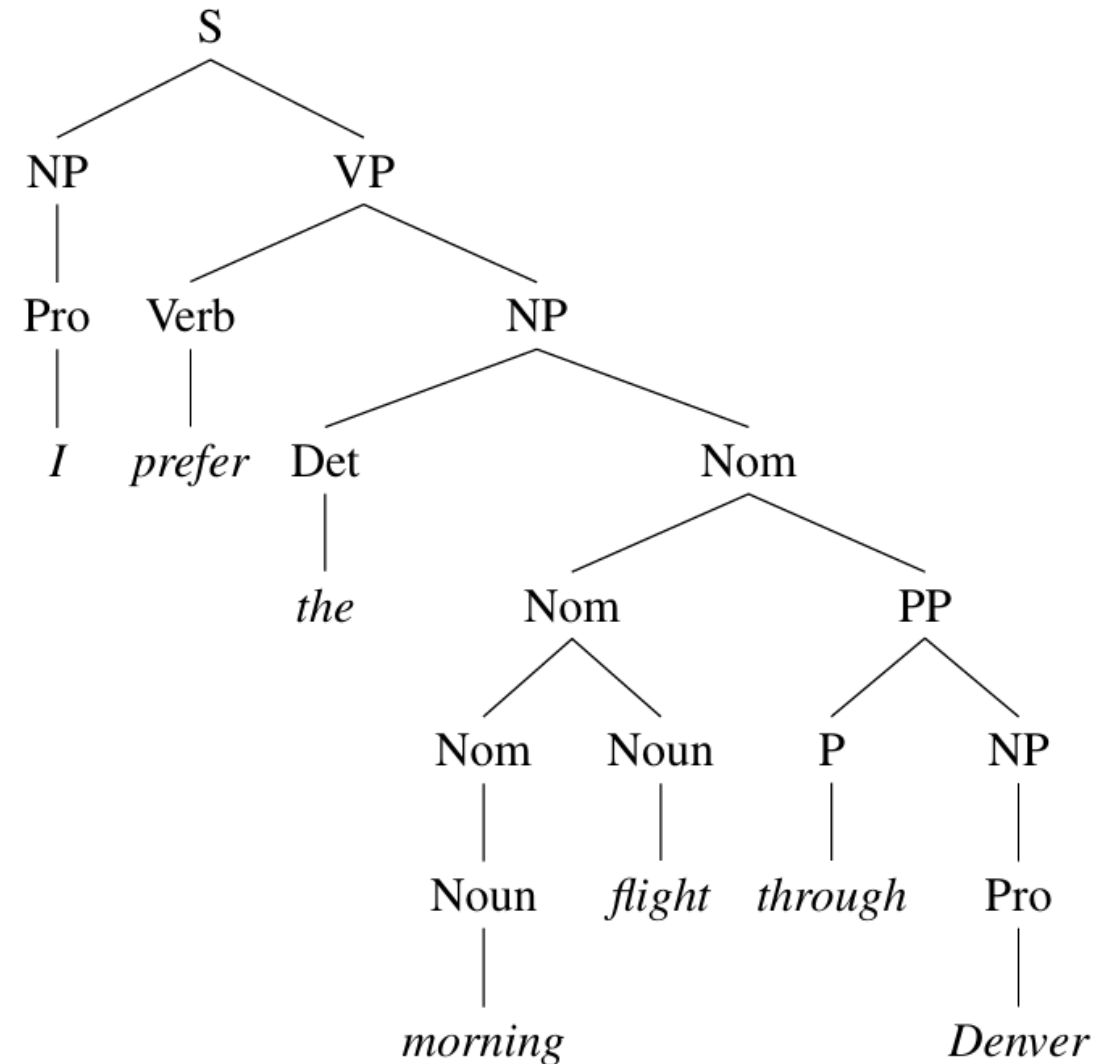
■ Ngữ pháp thành phần

(Constituency Grammars):

- Dựa trên các thành phần cụm từ và các quy tắc cấu trúc cụm từ (như NP - cụm danh từ, VP - cụm động từ).
- Phân chia câu thành các khối cấu trúc phân cấp lồng nhau.

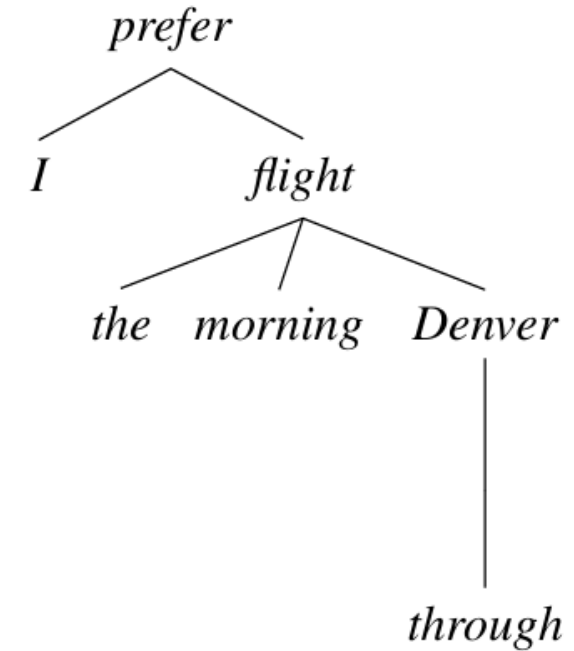
■ Một câu cơ bản (sentence) gồm:

- Noun phrase (NP)
 - Tiếp tục định nghĩa...
- Verb phrase (VP)
 - Gồm Verb + NP
 - ...

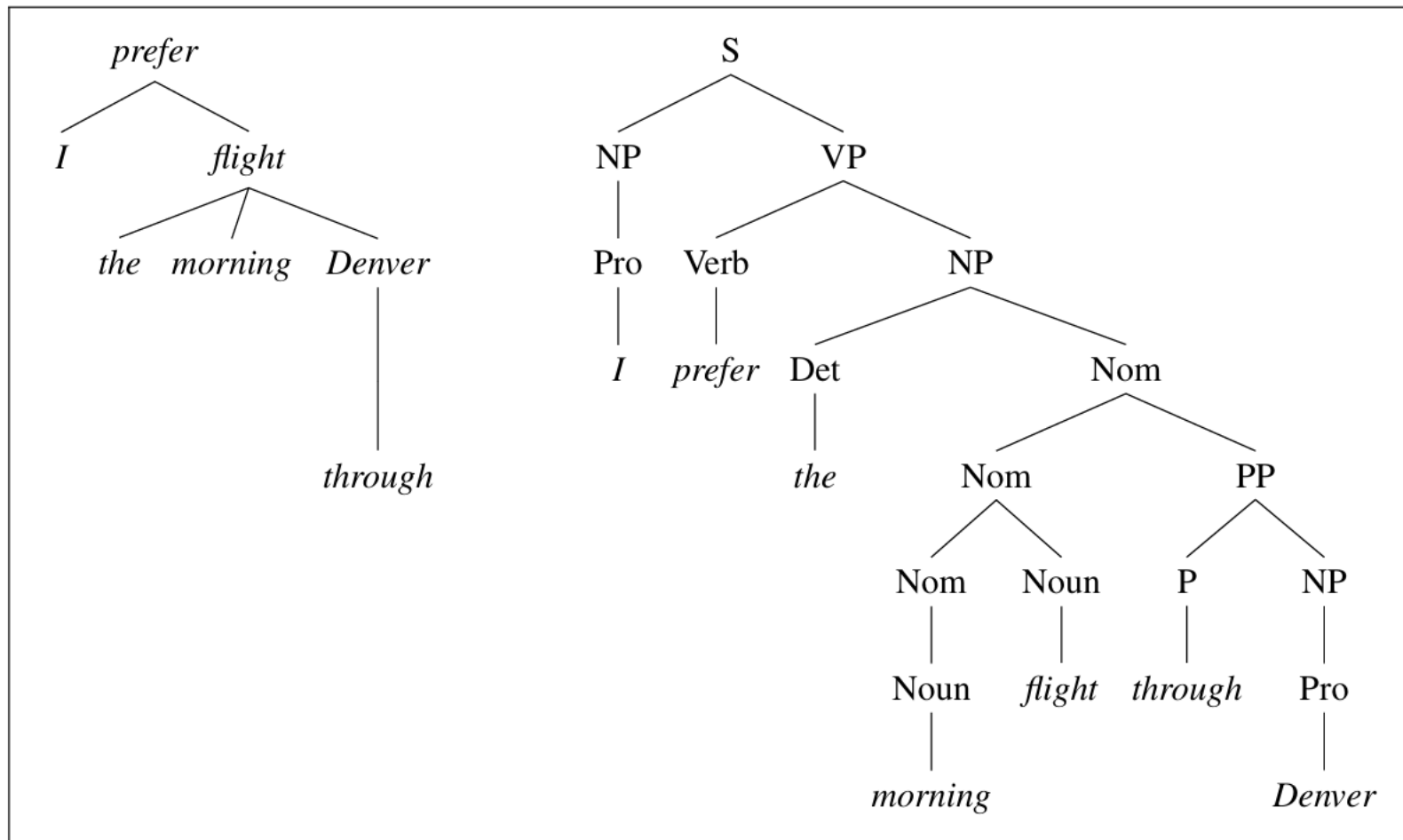


Dependency Grammar and Constituency Grammar

- **Ngữ pháp phụ thuộc (Dependency Grammars):**
 - Mô tả cấu trúc cú pháp của câu hoàn toàn dựa trên các quan hệ ngữ pháp nhị phân có hướng giữa các từ.
 - Không sử dụng các thành phần cụm từ (**phrasal constituents**) hay các quy tắc cấu trúc cụm từ.
 - Cấu trúc này bao gồm một từ chính (**head**) và một từ phụ thuộc (**dependent**) liên kết với nhau bằng các cung (**arc**) có nhãn (**label**).
- Hiện nay Ngữ pháp phụ thuộc **phổ biến hơn** Ngữ pháp thành phần trong Xử lý ngôn ngữ tự nhiên.



- Phân tích phụ thuộc và thành phần (Dependency and constituent analyses) cho câu “I prefer the morning flight through Denver”

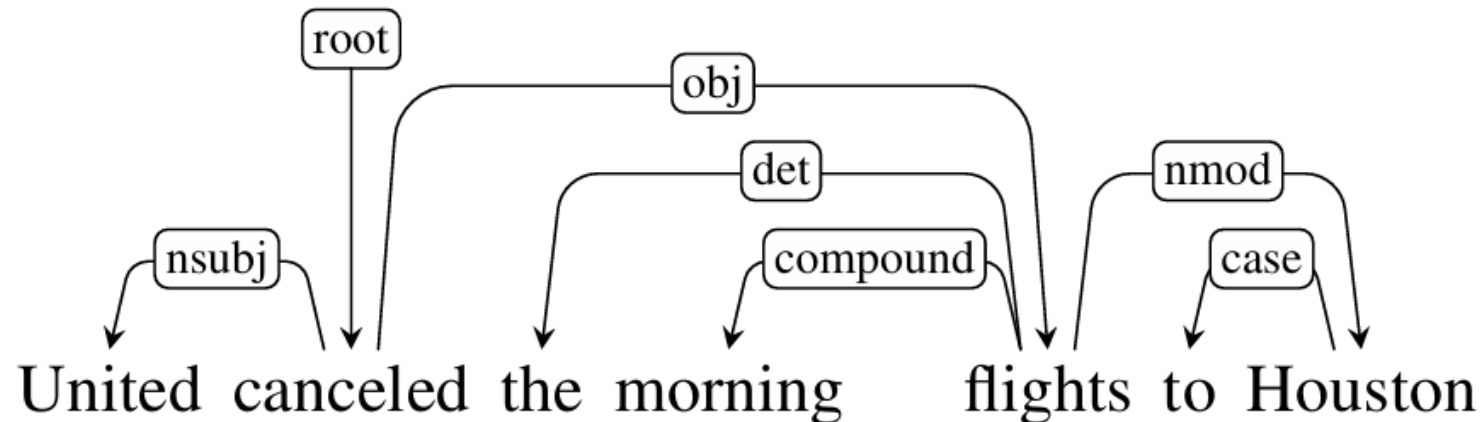


Quan hệ phụ thuộc (Dependency Relations)

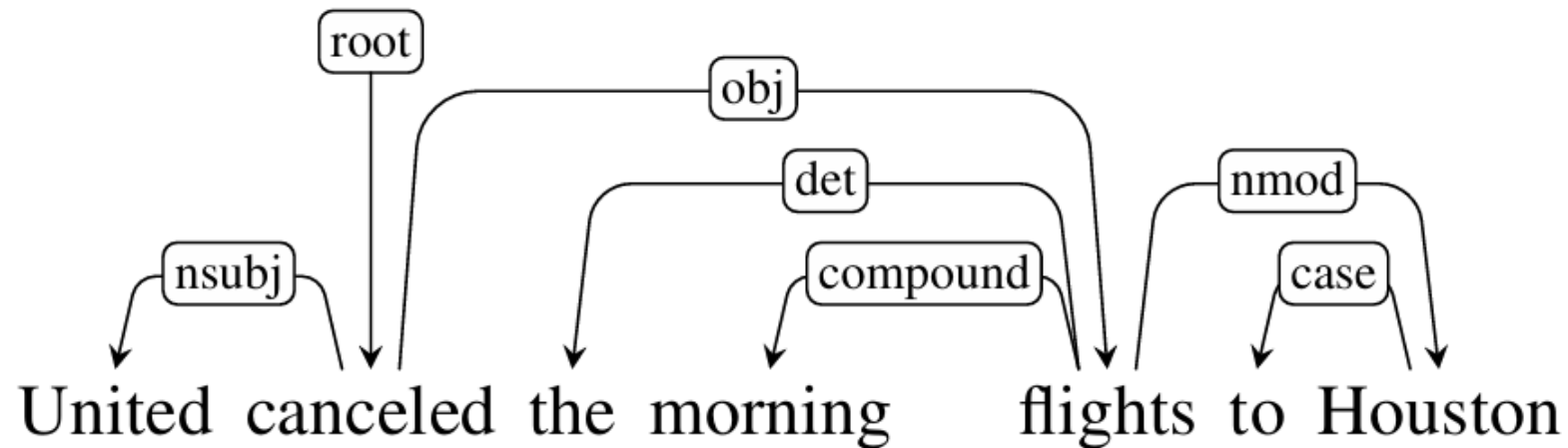
- Mô tả cấu trúc cú pháp của câu được xây dựng từ các mối liên kết trực tiếp giữa các từ.
 - Cấu trúc nhị phân: mỗi quan hệ phụ thuộc bao gồm một từ chính (**head**) đóng vai trò từ tổ chức trung tâm (**central organizing word**) và một từ phụ thuộc (**dependent**) đóng vai trò bổ nghĩa (**modifier**).
 - Quan hệ/Chức năng ngữ pháp (**grammatical relation/function**): các quan hệ này phân loại chức năng ngữ pháp mà từ phụ thuộc đảm nhiệm đối với từ chính, như chủ ngữ (**nsubj**), tân ngữ trực tiếp (**obj**), tân ngữ gián tiếp (**iobj**)...
- Ưu điểm của quan hệ phụ thuộc là có khả năng xử lý các ngôn ngữ có trật tự từ tự do (**free word order**)
 - Ví dụ: tính từ có thể tự do đứng trước hay sau danh từ

Dự án Universal Dependencies (UD)

- Cung cấp một danh mục gồm 37 quan hệ phụ thuộc (**dependency relations**) áp dụng thống nhất cho hơn 100 ngôn ngữ khác nhau.
- Tập các quan hệ thường dùng chia làm 2 tập hợp (**sets**):
 - Quan hệ mệnh đề (**Clausal relations**)
 - Mô tả vai trò cú pháp (**syntactic roles**) đối với một vị từ (**predicate**) (thường là động từ), ví dụ: nsubj, obj, ccomp
 - Quan hệ bổ nghĩa (**Modifier relations**)
 - Phân loại cách các từ bổ nghĩa (**modifier**) cho từ chính của chúng, ví dụ: nmod (bổ nghĩa danh từ), amod (bổ nghĩa tính từ), det (từ hạn định)



Dự án Universal Dependencies (UD)



- Ở đây, các quan hệ mệnh đề (**clausal relations**) NSUBJ và OBJ xác định chủ ngữ (**subject**) và tân ngữ trực tiếp (**direct object**) của vị từ (**predicate**) *cancel*
- Trong khi các quan hệ NMOD, DET và CASE biểu thị các bổ ngữ (**modifiers**) của các danh từ (**nouns**) *flights* và *Houston*

Dự án Universal Dependencies (UD)

Clausal Argument Relations	Description
NSUBJ	Nominal subject
OBJ	Direct object
IOBJ	Indirect object
CCOMP	Clausal complement
Nominal Modifier Relations	Description
NMOD	Nominal modifier
AMOD	Adjectival modifier
APPOS	Appositional modifier
DET	Determiner
CASE	Prepositions, postpositions and other case markers
Other Notable Relations	Description
CONJ	Conjunct
CC	Coordinating conjunction

Figure 19.2 Some of the Universal Dependency relations ([de Marneffe et al., 2021](#)).

Dự án Universal Dependencies (UD)

Relation	Examples with <i>head</i> and dependent
NSUBJ	United <i>canceled</i> the flight.
OBJ	United <i>diverted</i> the flight to Reno. We <i>booked</i> her the first flight to Miami.
IOBJ	We <i>booked</i> her the flight to Miami.
COMPOUND	We took the morning <i>flight</i> .
NMOD	<i>flight</i> to Houston .
AMOD	Book the cheapest <i>flight</i> .
APPOS	<i>United</i> , a unit of UAL, matched the fares.
DET	The <i>flight</i> was canceled. Which <i>flight</i> was delayed?
CONJ	We <i>flew</i> to Denver and drove to Steamboat.
CC	We flew to Denver and <i>drove</i> to Steamboat.
CASE	Book the flight through <i>Houston</i> .

Figure 19.3 Examples of some Universal Dependency relations.

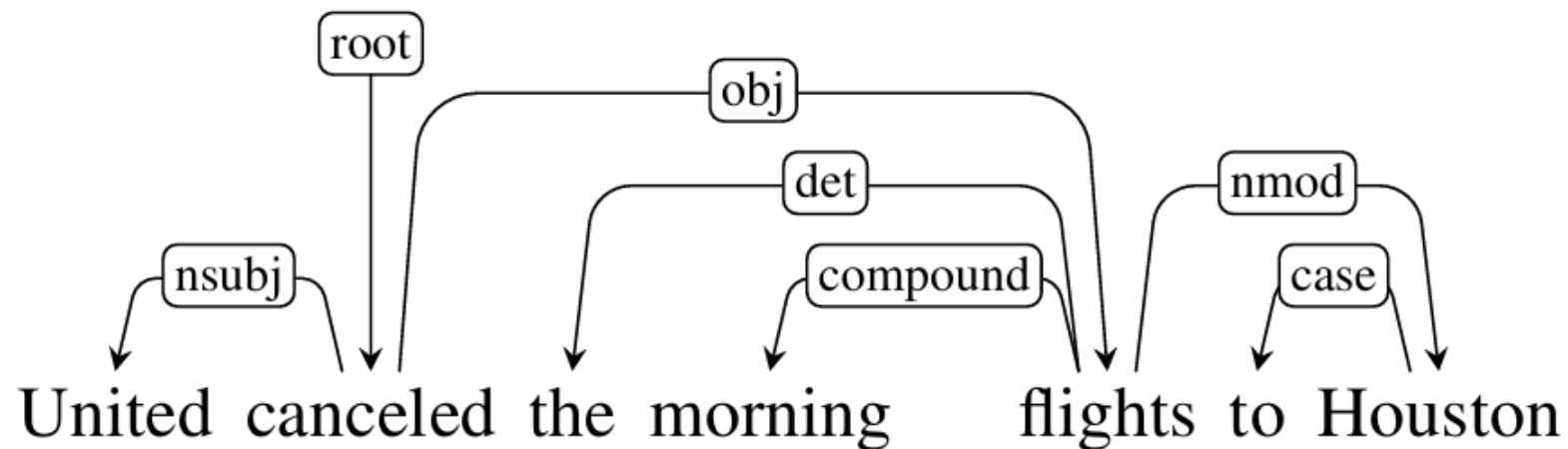
Dependency Formalisms

- Cấu trúc phụ thuộc có thể được biểu diễn dưới dạng đồ thị có hướng
 $G = (V; A)$
 - V : tập hợp các đỉnh (các từ trong câu)
 - A : tập hợp các cặp đỉnh (cung) có thứ tự, mô tả các quan hệ chức năng ngữ pháp (**grammatical function relationships**) giữa **head-dependent**
- Một cấu trúc phụ thuộc được coi là một cây phụ thuộc (**dependency tree**) hợp lệ nếu thỏa mãn ba điều kiện:
 - Có duy nhất một nút gốc (**single designated root node**) không có cung đi vào.
 - Mọi nút khác (trừ nút gốc) đều có đúng một cung đi vào (**exactly one incoming arc**).
 - Có một đường đi duy nhất (**unique path**) từ nút gốc đến mọi nút khác trong cây.

Dependency Formalisms

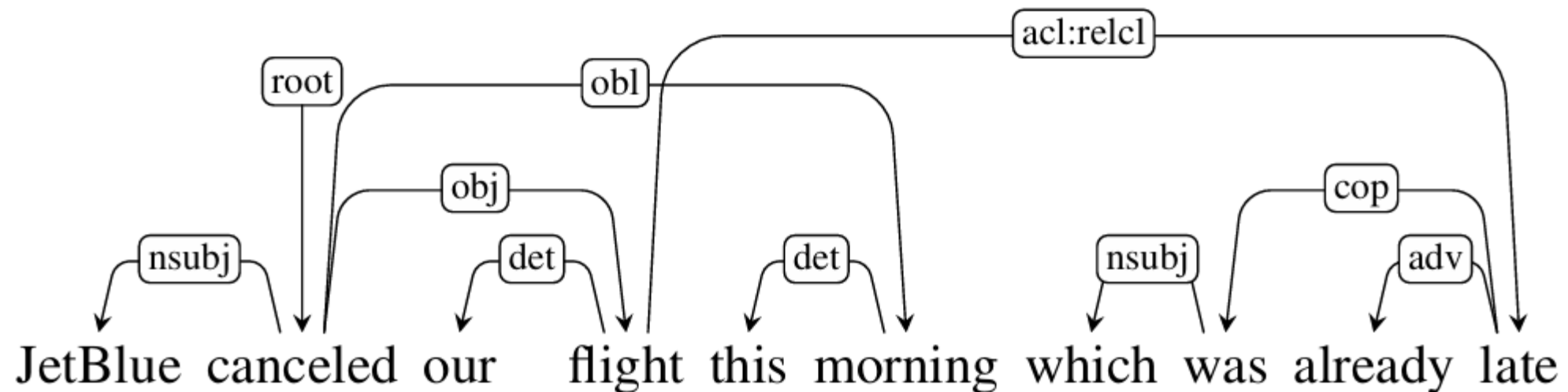
■ Tóm lại, những ràng buộc này đảm bảo:

- mỗi từ chỉ có một từ chính (**head**),
- cấu trúc phụ thuộc được liên thông,
- và có một nút gốc (**root**) duy nhất mà từ đó người ta có thể theo một đường dẫn có hướng duy nhất đến từng từ trong câu.



Tính dự chiếu (Projectivity)

- Tính dự chiếu (**projectivity**) là một ràng buộc (**constraint**) quan trọng trong cấu trúc cây phụ thuộc, liên quan đến trật tự các từ trong câu.
 - Một cung (**arc**) nối từ từ chính (**head**) đến từ phụ thuộc (**dependent**) được coi là dự chiếu nếu có một đường đi từ từ chính đó đến mọi từ nằm giữa nó và từ phụ thuộc trong câu.
 - Một cây phụ thuộc được gọi là dự chiếu nếu nó có thể được vẽ mà không có các cạnh nào chồng chéo (**crossing edges**) lên nhau.



-
- JetBlue canceled our flight this morning which was already late

Tính dự chiếu (Projectivity): ý nghĩa thực tế

■ Sự phổ biến

- Hầu hết các ngân hàng cây (**treebanks**) tiếng Anh được xây dựng dựa trên các cấu trúc dự chiếu.
- Tuy nhiên, các cấu trúc phi dự chiếu lại xuất hiện rất phổ biến trong các ngôn ngữ có trật tự từ tự do.

■ Hạn chế của thuật toán

- Các thuật toán phân tích cú pháp dựa trên chuyển trạng thái (**transition-based**) cơ bản chỉ có thể tạo ra các cây dự chiếu, dẫn đến việc sẽ gặp lỗi khi xử lý các câu có cấu trúc như ví dụ trên.
- Ngược lại, các phương pháp dựa trên đồ thị (**graph-based**) có khả năng xử lý và tạo ra các cây phi dự chiếu một cách linh hoạt hơn.

Ngân hàng cây phụ thuộc (Dependency Treebanks)

- Mô tả các kho dữ liệu văn bản đã được gán nhãn cấu trúc cú pháp phụ thuộc.
- Đóng vai trò quan trọng trong xử lý ngôn ngữ tự nhiên
 - Huấn luyện (Training)
 - Cung cấp dữ liệu đầu vào để các bộ phân tích cú pháp (**parsers**) học cách gán nhãn quan hệ giữa các từ.
 - Đánh giá (Evaluation)
 - Đóng vai trò là "nhãn chuẩn" (**gold labels**) để đo lường độ chính xác của các thuật toán phân tích cú pháp.
 - Nghiên cứu ngôn ngữ học
 - Cung cấp dữ liệu thực tế cho các nghiên cứu về ngôn ngữ học ngữ liệu (**corpus linguistics**)

Dependency Treebanks: phương pháp xây dựng

- Các ngân hàng cây có thể được tạo ra thông qua các cách sau:
 - Gán nhãn thủ công (**human annotators**)
 - Các chuyên gia ngôn ngữ (**annotators**) trực tiếp xây dựng cấu trúc phụ thuộc cho một tập ngữ liệu (**corpus**) hoặc hiệu đính thủ công (**hand-correcting**) các kết quả từ các bộ phân tích cú pháp tự động (**automatic parser**).
 - Chuyển đổi tự động
 - Sử dụng các quy tắc xác định để chuyển đổi các ngân hàng cây dựa trên cấu trúc thành phần (**constituent-based treebanks**) sẵn có sang dạng cấu trúc phụ thuộc.
 - Tuy nhiên, phương pháp này đôi khi có thể dẫn đến các lỗi về tính dự chiếu.

Dự án Universal Dependencies (UD)

<https://universaldependencies.org/>

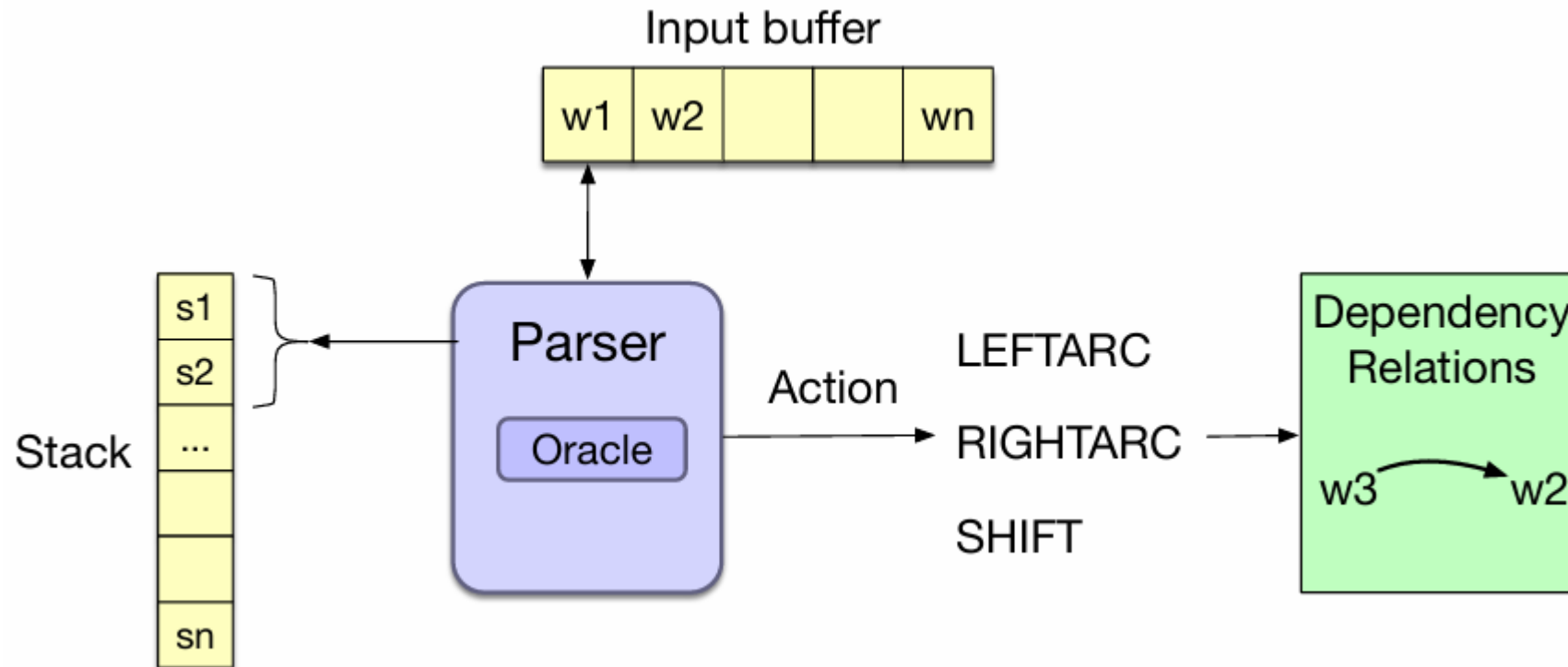
- Đây là dự án cộng đồng mở lớn nhất hiện nay nhằm chuẩn hóa việc gán nhãn phụ thuộc trên toàn thế giới:
 - Quy mô:
 - Hiện có gần 200 ngân hàng cây (**dependency treebanks**) bao phủ hơn 100 ngôn ngữ khác nhau.
 - Tiêu chuẩn:
 - UD cung cấp một danh mục gồm 37 quan hệ phụ thuộc chung, cho phép so sánh và phân tích cú pháp xuyên ngôn ngữ.
 - Ví dụ minh họa:
 - Tài liệu đưa ra các ví dụ về cấu trúc cây UD cho nhiều ngôn ngữ như tiếng Tây Ban Nha, tiếng Basque và tiếng Quan Thoại để minh họa tính đa dạng của hệ thống này.

Dự án Universal Dependencies (UD)

- Các thư viện và công cụ Phân tích cú pháp (Dependency Parsing) và trực quan hóa kết quả (Visualizers)
 - spaCy visualizers
 - <https://spacy.io/usage/visualizers/>
 - Dependency Parsing with NLTK
 - <https://www.geeksforgeeks.org/nlp/dependency-parsing-with-nltk/>
 - CoreNLP
 - <https://stanfordnlp.github.io/CoreNLP/demo.html>

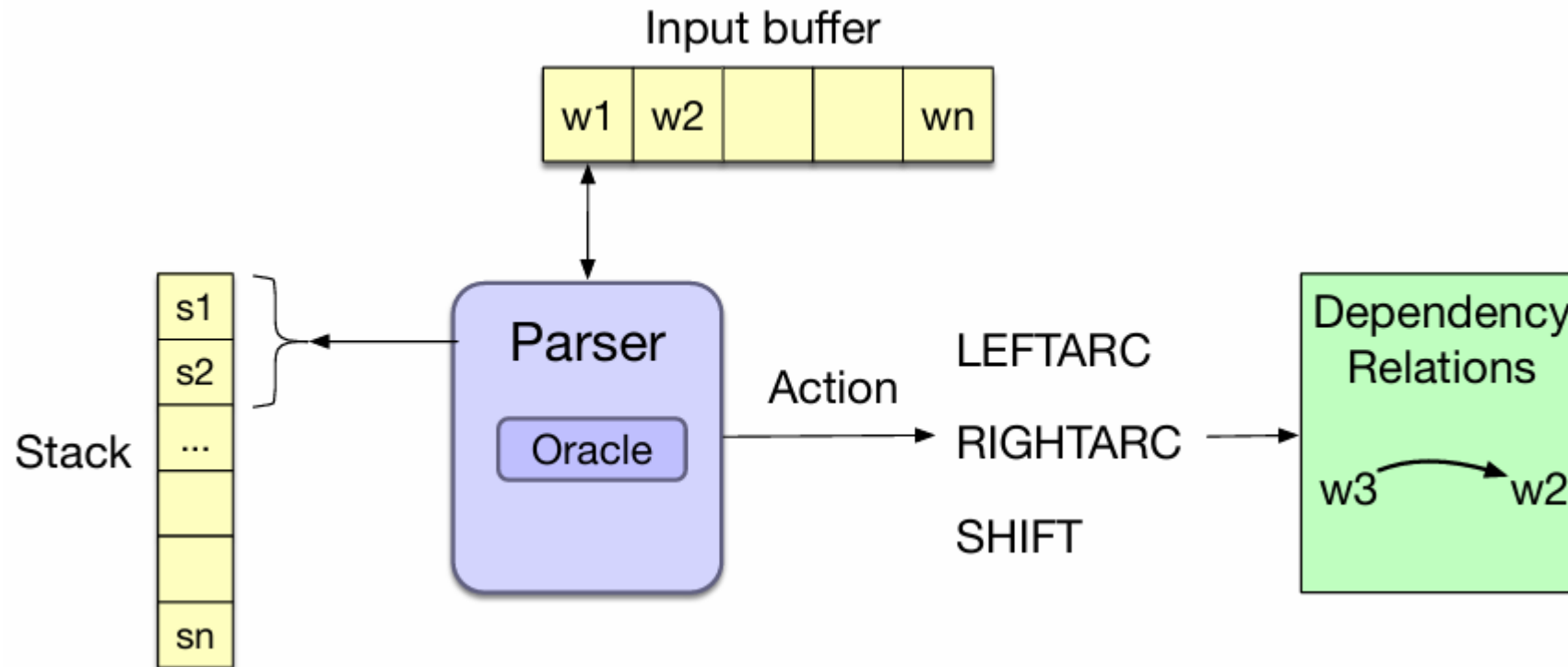
Transition-Based Dependency Parsing

- Phân tích cú pháp phụ thuộc dựa trên chuyển tiếp (Transition-Based Dependency Parsing)
 - Là phương pháp xây dựng cây phụ thuộc dựa trên mô hình **shift-reduce**, vốn ban đầu được phát triển để phân tích các ngôn ngữ lập trình.



Transition-Based Dependency Parsing

- Bộ phân tích cú pháp cơ bản dựa trên chuyển tiếp (**Basic transition-based parser**)
 - Parser kiểm tra 2 phần tử trên cùng của ngăn xếp (**stack**) và chọn một hành động (**action**) bằng cách tham khảo bộ dự đoán (**predictor/oracle**) kiểm tra cấu hình hiện tại.



Transition-Based Dependency Parsing

- Bộ phân tích cú pháp cơ bản dựa trên chuyển tiếp tổng quát (A generic transition-based dependency parser)

function DEPENDENCYPARSE(*words*) **returns** dependency tree

state $\leftarrow$ { [root], [*words*], [] } ; initial configuration

while *state* **not final**

t $\leftarrow$ ORACLE(*state*) ; choose a transition operator to apply

 state $\leftarrow$ APPLY(*t*, *state*) ; apply it, creating a new state

return *state*

Transition-Based Dependency Parsing

- Bộ phân tích cú pháp (**parser**) gồm:
 - Ngăn xếp (**stack**) σ
 - Bắt đầu với ký hiệu (**symbol**) **ROOT**
 - Bộ đệm (**buffer**) β
 - Bắt đầu với câu đầu vào (**input sentence**), chứa danh sách các từ (**tokens**) trong câu chưa được xử lý.
 - Tập hợp các quan hệ (**set of dependency arcs**) A
 - Đại diện cho các cạnh/cung của cây phụ thuộc đang được xây dựng dần dần.
 - Bắt đầu với tập rỗng $A = \emptyset$
 - Tập các hành động (**actions**) để chuyển trạng thái
 - **LEFTARC**: Tạo một quan hệ phụ thuộc w_1 (**head**) $\rightarrow w_2$ (**dependent**), trong đó w_1 là từ ở đỉnh ngăn xếp (**top**), w_2 là từ nằm ngay dưới w_1 ; sau đó loại bỏ từ w_2 khỏi ngăn xếp.
 - **RIGHTARC**: w_2 (**head**) $\rightarrow w_1$ (**dependent**); sau đó loại bỏ từ w_1 khỏi ngăn xếp.
 - **SHIFT**: lấy từ (**token**) ở đầu bộ đệm (**buffer**) và đẩy (**push**) vào ngăn xếp (**stack**)

Basic transition-based dependency parser

Start: $\sigma = [\text{ROOT}]$, $\beta = w_1, \dots, w_n$, $A = \emptyset$

1. Shift $\sigma, w_i | \beta, A \rightarrow \sigma | w_i, \beta, A$

2. Left-Arc_r $\sigma | w_i, w_j | \beta, A \rightarrow \sigma, w_j | \beta, A \cup \{r(w_j, w_i)\}$

3. Right-Arc_r $\sigma | w_i, w_j | \beta, A \rightarrow \sigma, w_i | \beta, A \cup \{r(w_i, w_j)\}$

Finish: $\beta = \emptyset$

Actions (“arc-eager” dependency parser)

Start: $\sigma = [\text{ROOT}]$, $\beta = w_1, \dots, w_n$, $A = \emptyset$

1. Left-Arc_r $\sigma | w_i, w_j | \beta, A \rightarrow \sigma, w_j | \beta, A \cup \{r(w_j, w_i)\}$

Precondition: $r'(w_k, w_i) \notin A$, $w_i \neq \text{ROOT}$

2. Right-Arc_r $\sigma | w_i, w_j | \beta, A \rightarrow \sigma | w_i | w_j, \beta, A \cup \{r(w_i, w_j)\}$

3. Reduce $\sigma | w_i, \beta, A \rightarrow \sigma, \beta, A$

Precondition: $r'(w_k, w_i) \in A$

4. Shift $\sigma, w_i | \beta, A \rightarrow \sigma | w_i, \beta, A$

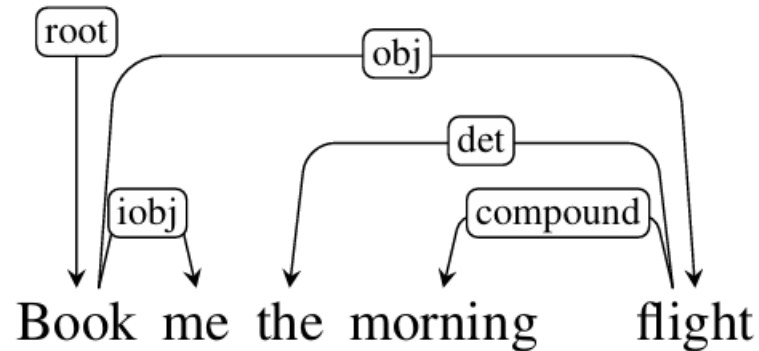
Finish: $\beta = \emptyset$

▪ Đây là biến thể “arc-eager” phổ biến:

- Một từ chính (head) có thể ngay lập tức chiếm lấy một từ phụ thuộc bên phải (right dependent), trước khi các từ phụ thuộc (dependents) của nó được tìm thấy.

Transition-Based Dependency Parsing: example

- Phân tích cú pháp phụ thuộc cho ví dụ sau:



- Trạng thái cấu hình sau khi đưa token ‘me’ vào ngăn xếp (**stack**)

Stack	Word List	Relations
[root, book, me]	[the, morning, flight]	

- Dựa trên **predictor/oracle** để chọn hành động RIGHTARC:
 - book (**head**) → me (**dependent**)
 - Lấy (**pop**) token ‘me’ ra khỏi stack, và tạo quan hệ (**relation**) book → me

Stack	Word List	Relations
[root, book]	[the, morning, flight]	(book → me)

Transition-Based Dependency Parsing: example

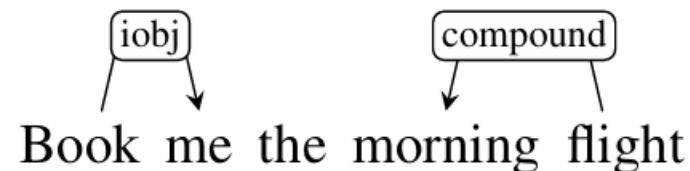
- Sau khi áp dụng một vài toán tử/hành động SHIFT (đưa các từ/token từ queue vào stack, nhưng predictor/oracle không dự đoán được)

Stack	Word List	Relations
[root, book, the, morning, flight]	[]	(book → me)

- LEFTARC

Stack	Word List	Relations
[root, book, the, flight]	[]	(book → me) (morning ← flight)

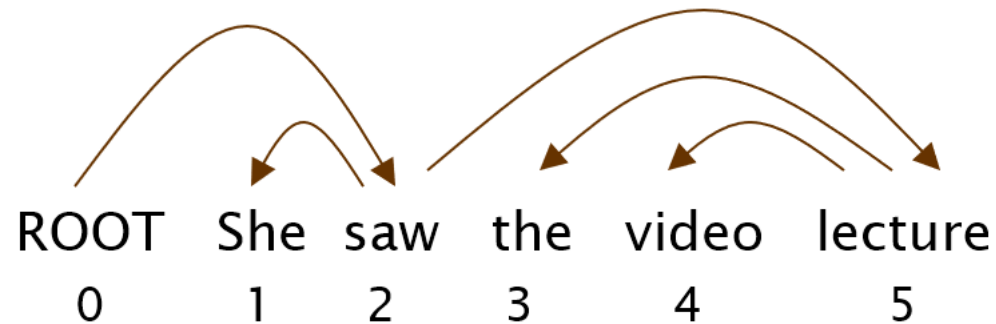
- Sau bước trên, phân tích cú pháp cho câu ví dụ gồm các cấu trúc sau:



Transition-Based Dependency Parsing: example

Step	Stack	Word List	Action	Relation Added
0	[root]	[book, me, the, morning, flight]	SHIFT	(book → me)
1	[root, book]	[me, the, morning, flight]	SHIFT	
2	[root, book, me]	[the, morning, flight]	RIGHTARC	
3	[root, book]	[the, morning, flight]	SHIFT	
4	[root, book, the]	[morning, flight]	SHIFT	
5	[root, book, the, morning]	[flight]	SHIFT	(morning ← flight) (the ← flight) (book → flight) (root → book)
6	[root, book, the, morning, flight]	[]	LEFTARC	
7	[root, book, the, flight]	[]	LEFTARC	
8	[root, book, flight]	[]	RIGHTARC	
9	[root, book]	[]	RIGHTARC	
10	[root]	[]	Done	

Evaluation of Dependency Parsing: (labeled) dependency accuracy



$$\text{Acc} = \frac{\text{\# correct deps}}{\text{\# of deps}}$$

$$\text{UAS} = 4 / 5 = 80\%$$

$$\text{LAS} = 2 / 5 = 40\%$$

Gold

1	2	She	nsubj
2	0	saw	root
3	5	the	det
4	5	video	nn
5	2	lecture	dobj

Parsed

1	2	She	nsubj
2	0	saw	root
3	4	the	det
4	5	video	nsubj
5	2	lecture	ccomp

Xây dựng bộ dự đoán (predictor/oracle)

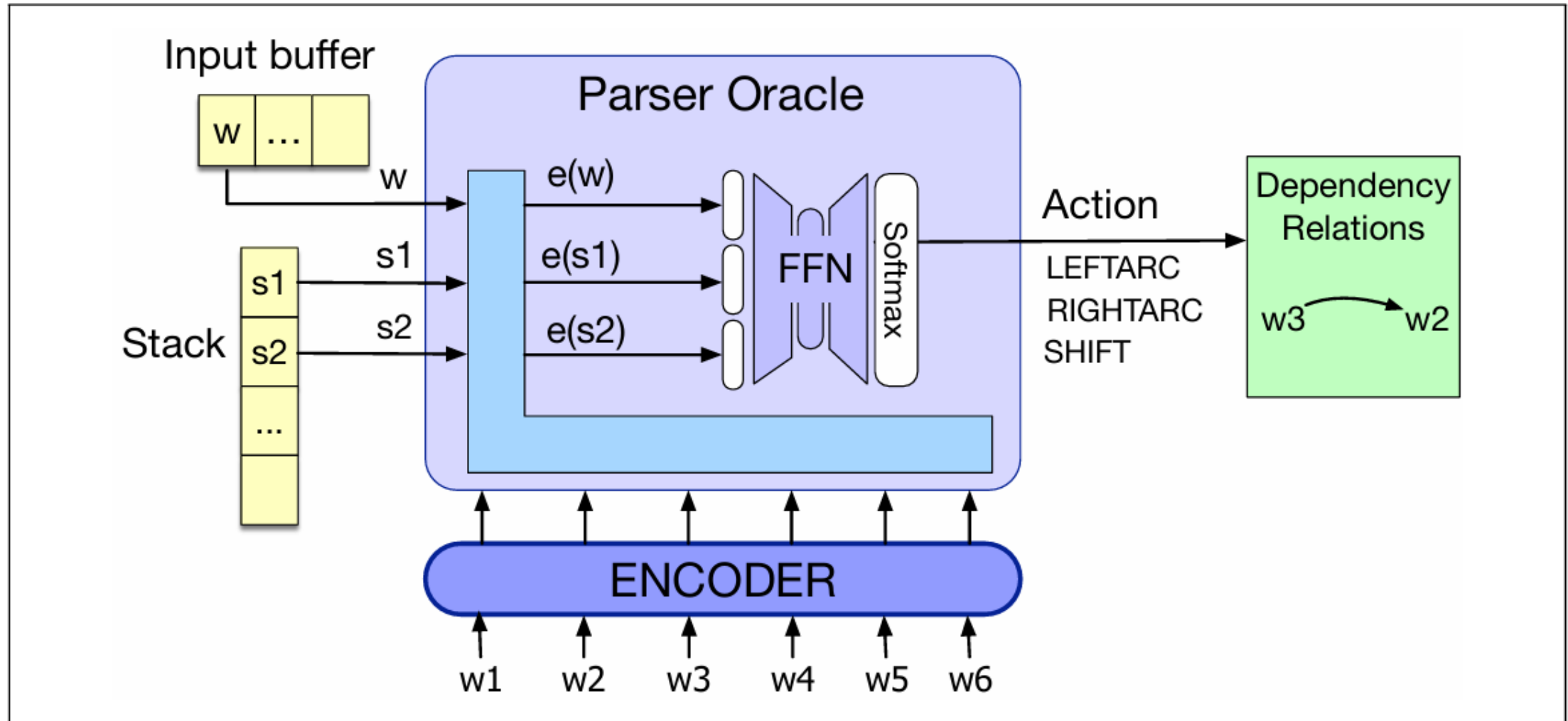
- Để lựa chọn các hành động/toán tử chuyển trạng thái trong phân tích cú pháp phụ thuộc
 - Bộ dự đoán (**oracle**) được huấn luyện thông qua học máy có giám sát (**supervised machine learning**)
 - Để thực hiện việc này, hệ thống cần dữ liệu huấn luyện bao gồm các cặp: cấu hình (**configuration**) và toán tử chuyển trạng thái tương ứng (**SHIFT, LEFTARC, hoặc RIGHTARC**)
 - Tuy nhiên, các ngân hàng cây (**treebanks**) thông thường chỉ cung cấp các câu văn đi kèm với cây phụ thuộc hoàn chỉnh của chúng, chứ không cung cấp sẵn chuỗi các bước chuyển trạng thái.
 - Phát sinh dữ liệu huấn luyện (**generate training data**)
 - Giả lập (**simulation**)

Xây dựng bộ dự đoán (predictor/oracle)

- Phát sinh các cấu hình (**configuration**) huấn luyện

Step	Stack	Word List	Predicted Action
0	[root]	[book, the, flight, through, houston]	SHIFT
1	[root, book]	[the, flight, through, houston]	SHIFT
2	[root, book, the]	[flight, through, houston]	SHIFT
3	[root, book, the, flight]	[through, houston]	LEFTARC
4	[root, book, flight]	[through, houston]	SHIFT
5	[root, book, flight, through]	[houston]	SHIFT
6	[root, book, flight, through, houston]	[]	LEFTARC
7	[root, book, flight, houston]	[]	RIGHTARC
8	[root, book, flight]	[]	RIGHTARC
9	[root, book]	[]	RIGHTARC
10	[root]	[]	Done

A neural classifier for oracle



Graph-Based Dependency Parsing

■ Phân tích cú pháp dựa trên đồ thị

- Họ thuật toán quan trọng thứ hai trong phân tích cú pháp phụ thuộc
- Chính xác hơn các bộ phân tích cú pháp dựa trên chuyển tiếp (transition-based parsers)

■ Phương pháp transition-based

- Thực hiện các quyết định cục bộ tham lam (**greedy**)
- Khó khăn khi từ chính (**head**) và từ phụ thuộc (**dependent**) nằm xa nhau (câu dài)

■ Phương pháp graph-based

- Phương pháp này tiếp cận bằng cách xem xét và chấm điểm trên toàn bộ cấu trúc cây.
- Chính xác hơn các bộ transition-based, đặc biệt trên các câu dài.

■ Nguyên lý

- Tìm kiếm trong không gian tất cả các cây cú pháp → tìm ra một/vài cây tối ưu hóa điểm số (**score**).

Graph-Based Dependency Parsing

▪ Tính điểm dựa trên cạnh (Edge-Factored):

- Để đơn giản hóa bài toán tìm kiếm, hệ thống thường giả định rằng điểm số của một cây bằng tổng điểm số của tất cả các cạnh cấu thành nên cây đó

$$Score(t, S) = \sum_{e \in t} Score(e)$$

- Do đó, bài toán gồm hai bước chính:
 - Gán điểm số cho mọi cạnh có thể có giữa các từ
 - Tìm cây khung tốt nhất dựa trên các điểm số đó

▪ Thuật toán Cây khung cực đại (Maximum Spanning Tree - MST)

- Thuật toán Chu-Liu/Edmonds (CLE)
 - Đây là thuật toán phổ biến nhất để tìm MST trong đồ thị có hướng.

Graph-Based Dependency Parsing

- Đồ thị có hướng (directed graph), có gốc ban đầu cho câu “Book that flight”

